Incident Triage RAG with LangChain

Colin Vitols | IT7075 Applied AI for Cybersecurity | November 2, 2026

Project folder: GitHub - langchain-rag-project / langchain_project

Video: Watch the recorded notebook walkthrough

Concept check

What problem does LangChain solve

With a direct API call, I still have to write the code that gets the documents, finds relevant text, and puts the prompt together. LangChain gives me reusable pieces for those steps, so I can spend more time on how the system works and less time connecting everything myself.

Why use a prompt template

A prompt template lets me write the instructions once and leave placeholders for things that change, like the question and retrieved context. That keeps the prompt consistent across questions and makes it easier to update without changing the rest of the pipeline.

What the prompt model parser chain does

The prompt fills in the question and context, the model processes those messages, and the parser turns the response into a string I can use. In this project, the offline model step quotes the retrieved text instead of generating a new answer.

How a retriever fits into a chain

A retriever takes the question and returns the document chunks that match it, along with information about where they came from. My chain uses those chunks as context and passes the original question into the prompt alongside them.

When to branch and when to run in parallel

I would use a branch when only one path should run, such as sending a critical security incident to urgent review and a routine issue to normal triage. I would use parallel steps when I need both results, such as reviewing endpoint evidence and network evidence separately before combining the findings.

The rebuild and what it replaced

For this project, I rebuilt my earlier incident-triage RAG with LangChain. I kept the same five Markdown documents and eight questions: six the documents can answer and two they cannot. The notebook checks that the copied files match the originals exactly and saves their SHA-256 hashes in data_manifest.json. As before, SOURCES.md stays out of the retrieval index.

The first part of the notebook works through the five practice stages using the course knowledge base. Then I apply those stages to my own documents using TextLoader, CharacterTextSplitter, local MiniLM embeddings, Chroma, and an LCEL chain. It runs on CPU without an API key. The answer step returns a labeled excerpt when it finds context and a refusal when it does not.

LangChain stage	Earlier code replaced
TextLoader and CharacterTextSplitter	load_and_chunk() and split_into_chunks()
HuggingFaceEmbeddings and Chroma.from_documents()	embed() and build_collection()
as_retriever() with score threshold	retrieve() and its distance-filtering loop
ChatPromptTemplate	SYSTEM.format(...) inside ask()
LCEL dictionary and pipe sequence	The orchestration inside ask()

I still need a small helper, format_sources(), to put source names into the context. The offline RunnableLambda also takes the place of grounded_answer(). The main change is that the sequence inside ask() is now written as a chain that I can run with invoke().

Where the framework helped and hid details

The most useful part is that LangChain carries the source information along with the text and lets the retriever plug directly into the chain. I no longer have to handle each embedding and database call myself. The tradeoff is that a short constructor call can hide details like the distance metric, score conversion, and exact chunk boundaries.

Those details matter when a question misses the evidence I expected it to find. I checked the raw distances against the relevance scores and compared the two retrievers on identical chunks. Each build also gets a unique collection name so running a cell again does not mix old chunks with new ones.

Measured comparison

To keep the comparison fair, I used the same normalized MiniLM embeddings, cosine distance, and scoring rule in both versions. A question counts as found when one retrieved chunk contains its expected evidence phrase. An unsupported question counts as refused when nothing is retrieved. These counts tell me how retrieval performed; they do not measure the quality of a generated answer.

The default uses 500-character chunks, an 80-character overlap, and k=3. The manual version keeps distances at or below 0.50, while LangChain keeps relevance scores at or above 0.50. Here, relevance = max(0, min(1, 1 - cosine distance)). The direction matters: tightening the old distance cutoff to 0.35 means raising the relevance cutoff to 0.65.

Change from default	Distance / score	Chunks M / L	Found M / L	Refused M / L
Default	0.50 / 0.50	26 / 26	6/6 | 6/6	2/2 | 2/2
Chunk size 350	0.50 / 0.50	41 / 39	5/6 | 5/6	2/2 | 2/2
Chunk size 700	0.50 / 0.50	20 / 20	4/6 | 4/6	2/2 | 2/2
k = 1	0.50 / 0.50	26 / 26	5/6 | 5/6	2/2 | 2/2
k = 5	0.50 / 0.50	26 / 26	6/6 | 6/6	2/2 | 2/2
Stricter threshold	0.35 / 0.65	26 / 26	2/6 | 2/6	2/2 | 2/2
Looser threshold	0.65 / 0.35	26 / 26	6/6 | 6/6	1/2 | 1/2
Loosest threshold	0.80 / 0.20	26 / 26	6/6 | 6/6	1/2 | 1/2

M = manual implementation; L = LangChain. Every row changes at most one setting from the default; overlap stays at 80 characters. The labels in results_langchain.csv preserve the earlier distance labels, while the explicit score_threshold column gives the LangChain value.

Both versions found all six answerable questions and refused both unsupported questions at the default settings. The notebook also reran all eight manual configurations and matched the earlier results. On the same default chunks, both retrievers returned the same text for every question, so the matching totals were not just a coincidence.

One difference showed up with 350-character chunks: my manual splitter produced 41 chunks, while LangChain produced 39. The two extra manual chunks were short pieces at the ends of documents that were already covered by other chunks. Leaving them out did not change the results. At the default size, both versions produced 26 chunks.

Making the cutoff stricter dropped the found count to 2/6. Looser cutoffs still found all six answers, but they also let Q8 through, leaving only one correct refusal. LangChain made the pipeline easier to connect, but it did not improve these retrieval results on its own.

Failure analysis and next choice

Q3 at k equals one

The question asks: “What should an analyst preserve from a suspected phishing email?” At k=1, the first email_triage.md chunk scores 0.5963 and does not contain the exact gold phrase. The second chunk scores 0.5926 and contains “preserve the original message with full headers and attachment hashes,” but k=1 excludes it.

The first passage is still useful: it says to preserve the original message or an approved export so the headers, attachment hashes, URLs, and routing details are available. It misses the exact phrase the scoring function looks for, even though it covers much of the same action. I would count this as a miss under the lab metric, but I would not call the retrieved passage unrelated.

The two scores are close, and allowing three chunks instead of one brings the expected phrase back into the results. This showed me why it is worth reading what was retrieved instead of only looking at the total score. For a larger project, I would also check whether the response communicates the right action, while keeping the original phrase-based metric for this comparison.

The unsupported policy question

Q8 asks for an exact SIEM retention period that the documents never give. With a loose cutoff, the retriever still finds related incident-policy text, and the offline answer step quotes it. That is a more concerning failure: the text sounds relevant, but it does not answer the question.

Which version I would build next time

I would use LangChain again for an application I planned to keep building because the pieces are easier to connect and extend. I would still keep checks for chunk boundaries, scores, and refusals. The manual version is useful when I need to understand why something happened, since I can follow each embedding, query, and filtering step directly.

Limits of the evidence

This project ran locally on CPU with an offline answer step, so I did not test a live generative model. Finding the expected phrase somewhere in the retrieved chunks also does not mean the displayed excerpt includes the whole answer. The eight questions are useful for checking this small project, but I would need a larger, separate test set before making broader accuracy claims.

Sources and reproducibility

Assignment: 05_LangChain/assignments/project/MiniProject_FrameworkRAG.pdf. Baseline: rag_project at commit 4bd86df17707d1a43831d1067c6555b891137002. Results, per-question diagnostics, transcript, hashes, and package versions are saved in the linked project folder. The accompanying live-recording guide shows how to restart the kernel and execute the notebook during the video.

LangChain Chroma API reference and CharacterTextSplitter API reference. Implementation behavior was also checked in the installed packages and executed assertions.